

Tarea 1

Las preguntas 1 a 3 requieren un desarrollo computacional usando los códigos Python que se le entregaron, con modificaciones que tendrán que hacer ustedes. Lean bien los enunciados de las tres preguntas ya que están conectados entre sí.

Consideraremos aquí el problema

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^T Q x + c^T x$$

donde $Q \in \mathbb{R}^{n \times n}$ es una matriz simétrica y definida positiva y $c \in \mathbb{R}^n$.

En estos problemas, queremos estudiar el comportamiento del Método del Gradiente “clásico” y del Método Simple de Primer Orden, en versión básica y acelerada, aplicado a este problema.

Pregunta 1: (10 pts)

Revise el material sobre la convergencia del Método del Gradiente para el problema de arriba, en los apuntes del curso, sección 2.4.1. Lo mismo está revisado en el video complementario entregado. Como podrá ver, la velocidad de convergencia depende de $\kappa(Q) = \mu_{\max}/\mu_{\min}$, donde $\mu_{\max}$ y $\mu_{\min}$ son el mayor y menor valor propio de la matriz Q , respectivamente. Este número se conoce como el “condicionamiento” de Q y, como se explica en el material, está relacionado al grado de excentricidad de las curvas de nivel de la función cuadrática.

(Nota: Si tienen debilidades en conceptos de normas, valores propios y similares, especialmente normas de matrices, revisen la cápsula de video de repaso inicial que está disponible en el Canvas).

- a) Modifique el código Python del Método del Gradiente, que se entregó en clases, para que sea aplicable a la función de este problema (Ind: esto es más fácil de lo que parece, note qué diferencia hay entre la función de este problema y la usada como ejemplo en clases).
- b) Modifique el código Python del Método del Gradiente o use el código para el Método Simple de Primer Orden para el problema de mínimos cuadrados, para construir una nueva rutina que ejecute el Método Simple de Primer Orden para el problema de esta tarea (note que el código para el problema de mínimos cuadrados es “casi” lo que necesita). Para esto deberá calcular la constante de Lipschitz adecuada, investigue funciones de cálculo numérico de Python que le puedan servir para esto.
- c) Modifique también el código Python del Método Simple de Primer Orden Acelerado de Nesterov para el problema de mínimos cuadrados, para adaptarlo al problema de esta tarea.
- d) Complemente los tres códigos para que calculen, también, el condicionamiento de la matriz Q Revise todos los códigos entregados, en alguna parte eso ya se hace).

En esta pregunta, debe incluir las transcripciones de los códigos producidos en su respuesta a la tarea.

Pregunta 2: (10 pts)

Ahora vamos a ejecutar el método del Gradiente sobre un conjunto de prueba de problemas generados en forma aleatoria. Para esto, observe el programa `main.py`, que es el que controla la ejecución de los métodos. Modifique la semilla del generador de números aleatorios usando la fecha de nacimiento de algún integrante del grupo, en el formato DDMMAAAA. Ahora, configure el código sucesivamente para generar y correr el Método del Gradiente para problemas con $n = 10, 50, 100, 150, 200, 250, 500, 750, 1000, 2000, 4000$, hasta llegar a un error de $\epsilon = 0,0001$. Si es necesario, ajuste el límite de iteraciones también.

Para cada ejecución del Método del Gradiente en los problemas registre: condicionamiento de la matriz Q generada, número de iteraciones, tiempo de ejecución (que es desplegado al final por los códigos de clases), y cualquier otro antecedente del proceso de convergencia.

Puede ocurrir que tenga problemas de convergencia, especialmente para las instancias más grandes. En ese caso, ajuste los parámetros de la función `fminbnd`. También puede, si quiere, reemplazar el uso de la función `fminbnd` por una fórmula explícita para el linesearch. Esta fórmula, de hecho, existe en el caso de querer minimizar una función cuadrática (la puede encontrar en las referencias del curso).

Pregunta 3: (10 pts)

Ahora, ejecute el Método Simple de Primer Orden y también el Método Acelerado de Nesterov, sobre exactamente los mismos datos de entrada que en la Pregunta 2 (si usa la misma semilla y los mismos n , eso es lo que ocurrirá).

Para cada ejecución de ambos métodos en los problemas registre: condicionamiento de la matriz Q generada (ya lo tiene de la pregunta anterior, en realidad), número de iteraciones, tiempo de ejecución, y cualquier otro antecedente del proceso de convergencia. Puede ser que para n grande la convergencia se haga muy lenta, tendrán que tomar alguna decisión sobre cómo abordar esto o, simplemente, interrumpir la ejecución antes.

Compare las corridas de los tres métodos. Haga eso en forma explícita con gráficos que muestren la convergencia y también gráficos de algún indicador de convergencia y condicionamiento de Q . ¿Puede decir algo sobre el efecto del condicionamiento de Q en uno u otro? ¿Puede decir algo sobre las diferencias de tiempo de ejecución? Analice qué factores del problema pueden estar influyendo en las diferencias.

Pregunta 4: (12 pts)

Considere ahora la función que se mostró en clases, para ilustrar los métodos de Gradiente y Newton,

$$f(x) = \frac{1}{2}x^T Qx + c^T x + \alpha(5 - x_n)^4$$

donde $Q \in \mathbb{R}^{n \times n}$ es simétrica y definida positiva, $c \in \mathbb{R}^n$, α es constante.

- a) (4 pts) En este caso, $\nabla^2 f(x)$ no es una matriz constante, depende de x . Argumente que $\nabla f(x)$ NO tiene propiedad de Lipschitz globalmente en $\mathbb{R}^n$.
- b) (4 pts) Por otro lado, supongamos que “sabemos” que las iteraciones del método siempre estarán contenidas en una región $D = \{x : \|x\|_2 \leq R\}$, donde R es un valor fijo. Argumente que, en ese caso, Sí es posible definir un valor de constante de Lipschitz para $\nabla f(x)$ que sea válida en D .
- c) (4 pts) Esto de tener un valor de R con la propiedad indicada en la parte b), puede ser algo complejo. Sin embargo, le pedimos que argumente que si α es pequeño, podemos cometer un error grande al estimar R y aún así tener una eventual constante de Lipschitz “razonable”.

Pregunta 5 (8 pts):

- a) (4 pts) Muchos computadores tienen, por ejemplo, un procesador Intel, o de otro fabricante. Elijan el computador de uno de los integrantes del grupo e identifiquen el tipo de procesador. Busquen en Internet la información (no olvide poner la cita) que le permita determinar cuantos “flop” por segundo puede realizar ese procesador. Con esa información en mano, construyan una tabla que indique cuanto tiempo se requiere para una iteración del Método de Newton, que asumiremos (como explicamos en clases) necesita ejecutar n^3 flops, una iteración de un método Quasi-Newton, que asumiremos requiere n^2 flops, y una iteración del método de Gradiente, que asumiremos requiere n flops. En la tabla ponga su estimación del tiempo que se requiere para realizar una iteración de cada uno de esos métodos. Haga esta tabla para los siguientes valores de n , $n = 50, 250, 500, 1,000, 5,000, 20,000, 150,000, 2,000,000, 10,000,000, 100,000,000$. (Un problema de optimización con millones de variables se puede dar en Machine Learning, reconstrucción de imágenes u otras áreas). Desde luego estos cálculos están ignorando otras operaciones que

también requieren tiempo (estimar los gradientes y Hessianos, el “linesearch”) y el hecho que el procesador no necesariamente estará exclusivamente dedicado a su trabajo, pero se obtendrá, al menos, una idea aproximada de los tiempos necesarios y su dependencia en n para cada método.

- b) (4 pts) Ahora, busque información respecto al supercomputador más rápido existente en este momento (suele cambiar año a año, así que busque información actualizada al día de hoy). Una vez identificado, fíjese en la información de su capacidad de cálculo, en “flops” por segundo. En su solución indique de qué supercomputador se trata y cuáles son sus características (no olvide indicar las fuentes de su información), y repita los cálculos de la parte a) para este caso y compare. ¿Qué opina respecto al balance entre capacidad de cálculo (según número de núcleos) y la posibilidad de resolver un problema? (Si les interesa saber hacia dónde piensan algunos que podría llegar la capacidad de computación, en algún futuro, busquen en Internet lo que se dice sobre un “Matrioshka Brain”).

Pregunta 6 (10 pts):

Considere la siguiente función en n variables:

$$f(x) = \sum_{j=1}^{n-1} |x_j - x_{j+1}| + \|Ax - b\|_2^2$$

donde $A \in \mathbb{R}^{m \times n}$ y $b \in \mathbb{R}^m$. El primer término de esta función es lo que se conoce como una “medida de variación total”.

- a) (2 pts) ¿Es convexa esta función? Justifique todo con precisión.
- b) (6 pts) Determine $\partial f(x)$ en un punto cualesquiera del dominio de f . Para abordar esto, le va a ser útil la siguiente “regla de la cadena” para funciones no diferenciables: sea f una función convexa, si $p \in \partial f(x)$ y $g(x) = f(Bx)$, donde B es alguna matriz de dimensiones adecuadas, entonces $B^T p \in \partial g(x)$. Tendrá que encontrar explícitamente la matriz B adecuada.
- c) (2 pts) Determine el esfuerzo computacional, en función de n y m , en el que se incurre para determinar un elemento de $\partial f(x)$, para un x dado en el dominio de f .

Pregunta 7 (10 pts):

Considere el siguiente problema de optimización:

$$\begin{aligned} \min \quad & \left\{ \max_{i=1, \dots, m} \{ \alpha_i^T x - b_i \} + \sum_{j=1}^{n-1} (x_j - x_{j+1})^2 \right\} \\ \text{s.a.} \quad & \sum_{j=1}^n x_j \leq 1 \\ & x_j \geq 0, j = 1, \dots, n \end{aligned}$$

Donde $\alpha_i \in \mathbb{R}^n$, $i = 1, \dots, m$, y los b_i son escalares. Esta función es convexa.

- a) (6 pts) Determine $\partial f(x)$ en un punto cualesquiera del dominio del problema (para el primer término de la función objetivo, le puede ser útil graficar un caso en una variable, para entender mejor esa función “máx”).
- b) (4 pts) Queremos resolver este problema usando el Método de Subgradiente. Como el problema tiene algunas restricciones, se debe usar una versión proyectada del método. Describa en detalle todos los pasos del algoritmo, indicando, en particular, la forma de abordar la etapa de proyección.